High-Level Architecture

Overview

Shio Runtime is composed of a small number of independent Runtime systems.

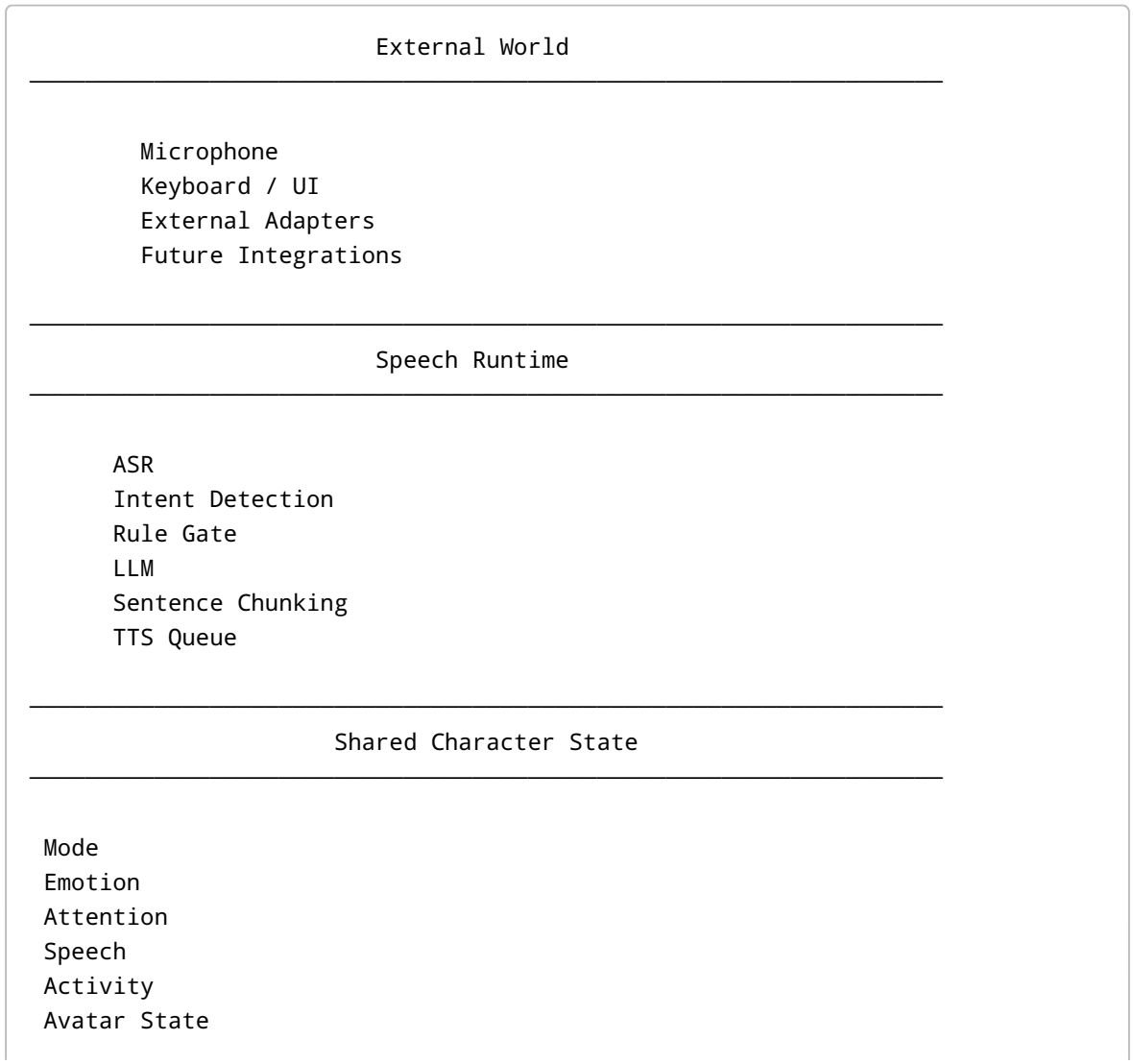
Each Runtime owns one responsibility.

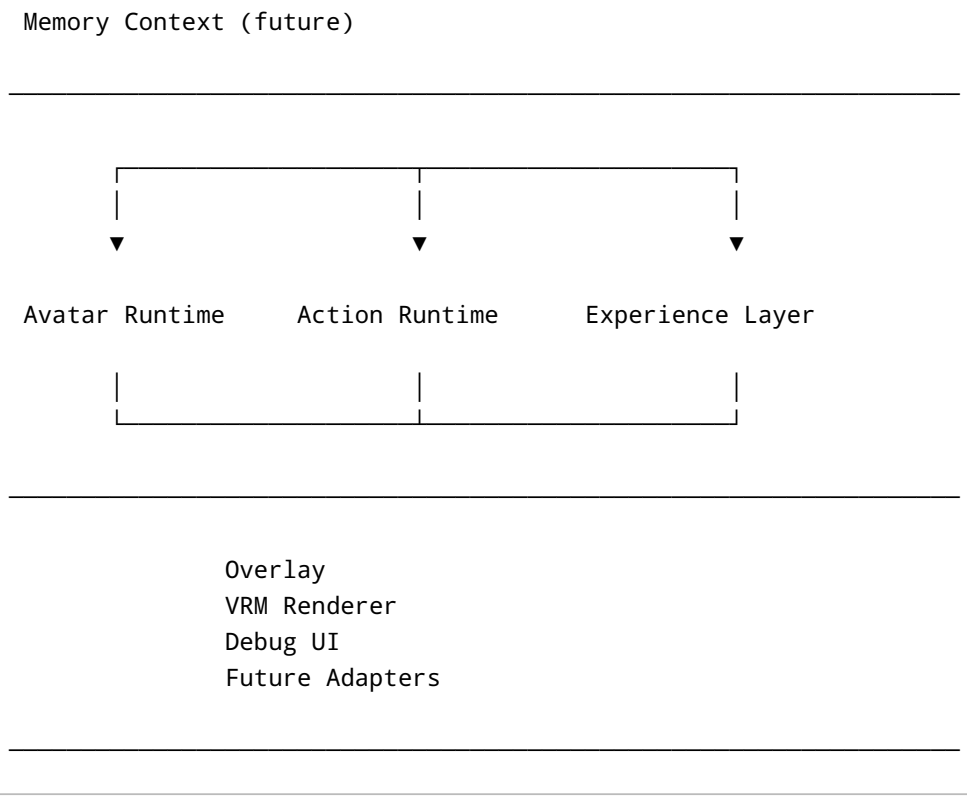
Together they coordinate every aspect of the digital character.

No Runtime is considered the "main controller" of the application.

The behavior of the character emerges from the cooperation of multiple Runtime systems rather than a single monolithic orchestrator.

Runtime Topology





Runtime Responsibilities

Each Runtime exists for a clearly defined purpose.

Runtime boundaries should remain stable throughout the lifetime of the project.

Changing implementation details is expected.

Changing Runtime responsibilities should be rare.

Speech Runtime

Responsible for verbal communication.

Responsibilities include:

- speech recognition;
- intent routing;
- LLM communication;
- streaming generation;
- sentence chunking;
- speech synthesis;
- conversation flow.

Speech Runtime knows nothing about:

- VRM;
- animations;
- rendering;
- streaming platforms;
- external integrations.

Its responsibility ends when the Character State has been updated.

Avatar Runtime

Responsible for the visual representation of the character.

Responsibilities include:

- animations;
- expressions;
- blinking;
- breathing;
- procedural motion;
- lipsync;
- idle behavior;
- VRM rendering.

Avatar Runtime never decides *what* the character should do.

It only decides *how* the current Character State should be visually represented.

Action Runtime

Responsible for executing actions in the outside world.

Responsibilities include:

- Tool Registry;
- Tool Execution;
- Permission Guard;
- Action Validation;
- External Commands.

Action Runtime never decides whether an action is appropriate.

It executes validated actions requested by higher-level systems.

Experience Layer

The Experience Layer transforms completed actions into believable character behavior.

Execution alone is not sufficient.

The character should emotionally experience the outcome of meaningful events.

Examples include:

- smiling after success;
- showing disappointment after failure;
- remaining silent;
- making a joke;
- reacting with surprise;
- intentionally ignoring an event.

This separation allows execution logic and character behavior to evolve independently.

Memory Layer (Future)

Memory Layer is responsible for preserving continuity across conversations.

Its responsibilities include:

- long-term memory;
- episodic memory;
- conversation summaries;
- user facts;
- future knowledge retrieval.

Memory Layer is intentionally independent from any specific storage implementation.

SQLite, PostgreSQL, vector databases or cloud storage are implementation details.

Runtime Communication

Runtime systems communicate through two mechanisms only.

Shared Character State

Long-lived observable information describing the current condition of the character.

Examples:

- current mode;

- emotion;
- attention;
- activity.

Events

Short-lived notifications describing something that has happened.

Examples:

- speech completed;
- tool finished;
- animation ended;
- user interrupted;
- sentence generated.

Events describe **what happened**.

Character State describes **what currently is**.

This distinction should remain consistent throughout the architecture.

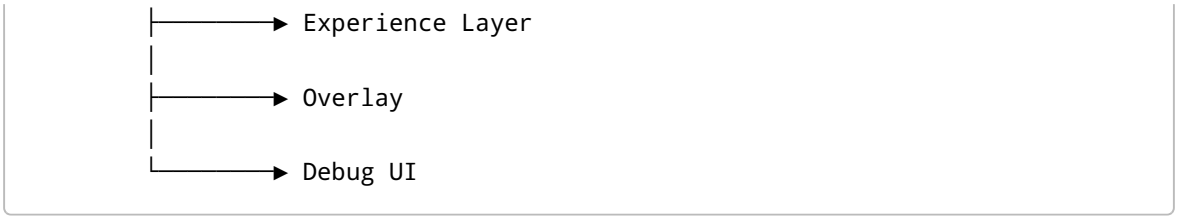
The Flow of Control

The architecture intentionally avoids a central controller.

Instead, responsibility flows naturally through Runtime systems.

A typical interaction looks like this:

```
User speaks
        │
        ▼
Speech Runtime
        │
        ▼
Rule Gate
        │
        ▼
LLM
        │
        ▼
Character State updated
        │
        ├──────────► Avatar Runtime
        │
```



No Runtime explicitly commands another Runtime to react.

Every Runtime independently observes the state relevant to its own responsibility.

This architecture minimizes coupling while maximizing extensibility.

Architectural Consequences

The Runtime architecture intentionally produces several desirable properties.

Replaceable Technologies

Any implementation may be replaced without redesigning the Runtime.

Examples include:

- LLM Provider;
- TTS Engine;
- ASR Engine;
- VRM Renderer;
- Desktop Framework.

Independent Evolution

Every Runtime can evolve independently.

Speech improvements do not require Avatar Runtime changes.

Avatar improvements do not require Action Runtime changes.

Memory improvements do not require Speech Runtime changes.

Scalable Architecture

Future Runtime systems can be introduced without redesigning existing Runtime boundaries.

Potential future Runtime systems include:

- Scheduler Runtime;
- Behavior Runtime;
- Memory Runtime extensions;
- Analytics Runtime;
- Streaming Runtime;
- Plugin Runtime.

Each should integrate through existing architectural contracts rather than introducing new direct dependencies.

The architecture grows by extending responsibilities, not by increasing coupling.